



SAPIENZA
UNIVERSITÀ DI ROMA

MACHINE LEARNING AND SECURITY

Lecture Notes

LECTURERS

Fabio De Gaspari

Dorjan Hitaj

Department of Computer Science
Sapienza University of Rome

NOTES PREPARED BY

Matt Clifford Trinidad Ramos

Contents

1	Course Introduction and Threat Modeling	1
1.1	Course Orientation	1
1.2	Why Machine-Learning Security Matters	2
1.3	Classical and Adversarial World Models	2
1.3.1	The classical view	2
1.3.2	The adversarial view	3
1.4	Motivating Example: Spam Filtering	3
1.5	The Cybersecurity Arms Race	4
1.6	Threat Modeling	4
1.6.1	Adversary's goal: integrity, availability, confidentiality	4
1.6.2	Adversary's knowledge	4
1.6.3	Kerckhoffs' principle	5
1.6.4	Adversary's capability: training time vs. inference time	5
1.6.5	Capability constraints and the feasible domain	6
2	Basic Machine-Learning Refresher	7
2.1	Types of Machine Learning	7
2.2	What It Means for a Program to Learn	7
2.2.1	Task	7
2.2.2	Performance measure	7
2.2.3	Experience	8
2.3	Features, Feature Space, and Data Representation	8
2.4	Linear Regression	9
2.4.1	Train, validation, and test sets	9
2.4.2	Mean squared error	10
2.5	Learning as Optimization	10
2.6	Gradient-Based Optimization	10
2.6.1	Derivative and local linear approximation	10
2.6.2	Single-variable gradient descent	11
2.6.3	Multivariable gradients	11

2.7	Generalization, Capacity, and Overfitting	11
2.8	Regularization	12
2.8.1	Weight decay	12
2.8.2	Dataset augmentation	13
2.9	Data Normalization	13
2.9.1	Min–max scaling	13
2.9.2	Z-score standardization	14
2.10	Validation and Cross-Validation	14
2.10.1	K-fold cross-validation	14
3	Decision Trees, Random Forests, and Deep Neural Networks	16
3.1	Decision Trees	16
3.1.1	Training a decision tree	16
3.1.2	Entropy as a purity measure	17
3.2	Ensemble Methods and Random Forests	17
3.2.1	Ensemble learning	18
3.2.2	Random Forest construction	18
3.2.3	Why Random Forests work well	18
3.3	Deep Neural Networks	18
3.3.1	What “deep” means	19
3.3.2	Function approximation	19
3.3.3	Feedforward composition and depth	19
3.4	Representation Learning in Feedforward Networks	20
3.5	Understanding the Neuron and Activation Function	20
3.5.1	Weighted inputs and a threshold	20
3.5.2	Activation functions	21
3.6	Forward Propagation	22
3.7	Gradient Descent for Neural Networks	23
3.8	Backpropagation	23
3.8.1	Error at the output layer	23
3.8.2	Propagating error to an earlier layer	24
3.8.3	Weight and bias gradients	24
3.9	Stochastic Gradient Descent	25
3.10	Loss Functions	25
3.10.1	Mean Squared Error	25

3.10.2	Binary cross-entropy	26
3.10.3	Multi-class cross-entropy	26
A	Notation and Quick Reference	28
A.1	Core Symbols	28
A.2	Formula Summary	28
B	Glossary	29
C	Source Coverage Map	30

Course Introduction and Threat Modeling

1.1 Course Orientation

The course is taught by **Fabio De Gaspari** and **Dorjan Hitaj**. Its central objective is to understand both sides of the relationship between machine learning and cybersecurity: the use of machine learning in security applications, and the vulnerabilities of machine-learning systems when they operate in adversarial contexts. The course also includes practical laboratories on selected topics.

Table 1.1: Course contacts and research focus.

Item	Information
Fabio De Gaspari	Assistant Professor, Department of Computer Science. Research focus: systems security; machine-learning security and privacy; network security. Homepage: https://sites.google.com/di.uniroma1.it/degaspari/home
Dorjan Hitaj	Assistant Professor, Department of Computer Science. Research focus: machine learning and cybersecurity. Homepage: https://sites.google.com/view/dorjanhitaj/
Google Classroom	https://classroom.google.com/c/MTYyNzMyNTQ2MTMx?cjc=tftag4vd ; course code: tftag4vd

Table 1.2: Lecture schedule and room exceptions.

Schedule item	Details
Tuesday	12:00–14:00, Aula 101, Palazzina D, Viale Regina Elena 295
Wednesday	10:00–13:00, Aula 4 De Lollis, Via Tiburtina 205
Exceptions	29/09/2026, 20/10/2026, and 10/11/2026: Aula 2L, Castro Laurenziano
Course material	Available in Classroom; mixture of slides and papers; no single textbook

Course logistics and assessment

Lectures are held on Tuesday and Wednesday. Course material is distributed through Google Classroom as a mixture of slides and papers rather than a single textbook. Assessment is centered on practical understanding: a course project implementing techniques from a covered topic, a possible class presentation depending on enrollment and timing, and an oral examination discussing the project and related material. The project idea should be confirmed with the lecturers before work begins, and the project should be submitted at least two weeks before the intended examination date.

The tentative syllabus includes:

- ▶ an introduction to machine learning and deep neural networks;
- ▶ evasion attacks and defenses;
- ▶ poisoning and trojanning attacks and defenses;
- ▶ certifiable robustness;
- ▶ generative models and large language models;
- ▶ inference attacks and federated learning;
- ▶ explainable AI.

1.2 Why Machine-Learning Security Matters

Modern machine-learning systems are embedded in voice assistants, autonomous-driving pipelines, large language models, and increasingly autonomous software agents. Their growing capability and economic value make robustness and security consequential, yet surprisingly small or inexpensive adversarial manipulations can sometimes induce incorrect behavior. Representative examples include adversarially modified traffic signs, adversarial audio targeting voice assistants, LLM jailbreaks, prompt-injection attacks, and security concerns around browser or agent systems. A demonstration resource used in the course is <https://adversarial-attacks.net/>. The central asymmetry is that a high-value, high-capability system may still be influenced by a comparatively cheap perturbation or crafted interaction.

Key Concept

A machine-learning model is not deployed in a passive world. Once a model influences real decisions, an attacker may deliberately choose inputs, training data, queries, or interactions *because of how the model behaves*. This invalidates the classical assumption that data generation is independent of the learned model.

1.3 Classical and Adversarial World Models

1.3.1 The classical view

A classical machine-learning pipeline can be represented as:

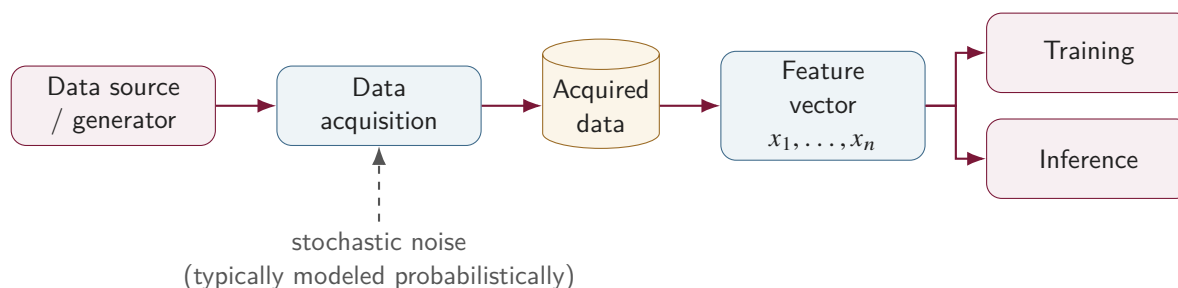


Figure 1.1: Classical world view: data are generated independently of the model, and acquisition noise is treated as stochastic.

The associated assumptions are that the **source of data does not depend on the model** and that acquisition noise is stochastic in nature, commonly modeled with distributions such as the Gaussian distribution.

1.3.2 The adversarial view

In an adversarial environment, the noise or manipulation is *purposeful*. The attacker can craft data based on knowledge of the model and its behavior.

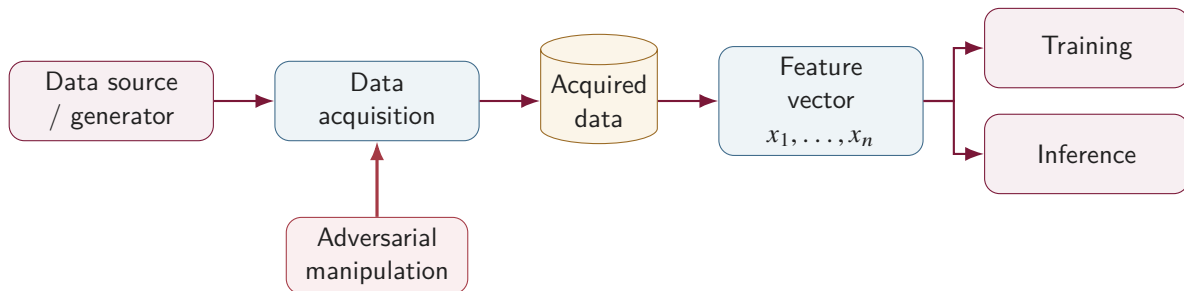


Figure 1.2: Adversarial world view: inputs may be deliberately manipulated to influence training or inference.

Stochastic noise vs. adversarial data

Stochastic noise is modeled as random variation. Adversarial data are **not random**: they are chosen with an objective and often depend on the decision rule, model parameters, available feedback, or other information about the system.

1.4 Motivating Example: Spam Filtering

A simple linear spam classifier shows how knowledge of a decision rule can change the data-generation process. Suppose a message contains the keywords *buy* and *bitcoins*, with weights 2.0 and 4.0. The classifier uses a threshold of 5.0.

$$s(x) = \sum_{j=1}^d w_j x_j. \quad (1.1)$$

where

- ▶ x_j : feature indicating the presence or amount of the j -th keyword;
- ▶ w_j : weight assigned to that feature by the classifier;
- ▶ d : number of features;
- ▶ $s(x)$: classifier score.

With *buy* and *bitcoins* present, the score is $2 + 4 = 6$, hence $6 > 5$ and the message is classified as spam. If the attacker inserts the word *uncle*, whose weight in this example is -2 , the score becomes $2 + 4 - 2 = 4$, so $4 < 5$ and the same underlying spam message crosses the decision boundary and is classified as non-spam.

Adversarial adaptation

The message is not modified by random corruption. It is modified *because* the attacker knows or infers which features reduce the spam score. The data-generation process therefore depends on the model.

1.5 The Cybersecurity Arms Race

Cybersecurity can be viewed as a repeated adversary–defender cycle:

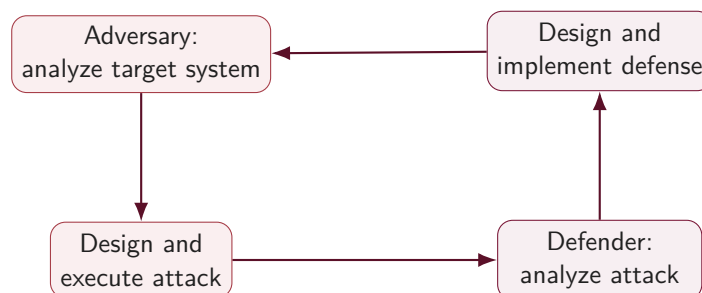


Figure 1.3: Security is an iterative arms race: attacks reveal weaknesses, defenses change the system, and attackers adapt.

Three practical rules follow: **know your adversary**, **be proactive**, and **protect your model**.

1.6 Threat Modeling

Definition

A machine-learning **threat model** characterizes the adversary along three dimensions:

1. **knowledge**: what the adversary knows about the data, model, and outputs;
2. **goal**: what security property the adversary wants to violate;
3. **capability**: what the adversary is able to modify or query, and under which constraints.

1.6.1 Adversary’s goal: integrity, availability, confidentiality

Adversarial goals can be mapped to the classical CIA security properties.

Table 1.3: Adversarial goals for machine-learning systems.

Goal	Meaning	Representative consequence
Integrity	Compromise the model’s intended operation	Deliberately cause samples to be misclassified
Availability	Compromise access to, or normal operation of, the model	Errors or behavior that can produce denial of service or prevent normal use
Confidentiality	Extract private information from the model	Reveal confidential information about the model or its users

1.6.2 Adversary’s knowledge

Adversarial knowledge can concern three major resources: **training data**, **model details**, and **output feedback**. Three broad attack settings are useful:

Table 1.4: Knowledge-based attack settings.

Setting	Knowledge	Interpretation
White-box	Perfect / complete	Worst-case scenario for the defender; provides an upper bound on performance degradation under attack.
Gray-box	Partial	Typical realistic setting; which information is available depends on the use case.
Black-box	No internal model knowledge in the simplified taxonomy used here	Very weak adversarial model and best case for the defender; this extreme is often unrealistic.

1.6.3 Kerckhoffs' principle

“The enemy knows the system”

A secure system should remain secure even when its design is public. Security should not rely on obscurity. In ML, it is often reasonable to assume that an attacker knows the algorithms being used, has at least partial information about training data, and can infer a rough model architecture. Therefore, an ML model should be tested under **varying levels of adversarial knowledge and access**.

1.6.4 Adversary's capability: training time vs. inference time

Two high-level capability surfaces are emphasized:

Table 1.5: Capabilities at training and inference time.

Phase	Adversarial capability	Representative examples
Training time	Modify or inject training data so that the learned model contains errors or vulnerabilities that appear later at inference	Poisoning, backdoors, payload injection
Inference time	Manipulate samples sent to the trained model or exploit model responses	Evasion attacks, adversarial examples, membership inference

Training-time manipulation: Microsoft Tay

Microsoft's Tay chatbot provides a concrete example of a learning system exposed to adversarial user-controlled data. A coordinated group exploited the system shortly after launch and drove it toward inappropriate outputs. The security lesson is broader than the product itself: when a system updates or adapts from user-provided interactions, the training signal becomes part of the attack surface.

1.6.5 Capability constraints and the feasible domain

Attackers are powerful but not unconstrained. Malicious modifications must often remain *valid* and *useful* in the underlying application domain:

- ▶ phishing emails must still be readable and understandable to humans;
- ▶ phishing-site images must still resemble the intended original sufficiently to be credible;
- ▶ malware binaries must remain executable and still perform their malicious function;
- ▶ an attacker typically does not control every stage of the entire ML pipeline.

At training time, the adversary may control only a small fraction of a very large dataset, and may be restricted to adding, removing, or modifying particular points. At inference time, query modifications must stay in the **feasible domain**: the subset of the input space that satisfies application-specific constraints, such as structural constraints on binary executables.

Do not choose an unrealistic attacker

Security analysis should avoid both extremes. Assuming an infinitely strong adversary can make the evaluation uninformative about real deployment, while assuming an unrealistically weak adversary can create a false sense of robustness. Systems should therefore be tested at **several levels of adversarial capability**.

Key Takeaways

- ▶ Classical ML assumes data generation is independent of the model; adversarial ML explicitly breaks that assumption.
- ▶ Threat modeling requires a clear statement of the adversary's **knowledge**, **goal**, and **capability**.
- ▶ Integrity, availability, and confidentiality provide a useful taxonomy of attacker objectives.
- ▶ White-, gray-, and black-box settings differ in how much the attacker knows about the system.
- ▶ Training-time and inference-time attacks expose different parts of the pipeline.
- ▶ Real attackers operate under feasibility constraints; robust evaluation should model these constraints rather than ignoring them.

Basic Machine-Learning Refresher

2.1 Types of Machine Learning

Three broad learning paradigms are:

- ▶ **supervised learning**: task-driven learning from labeled examples, typically for prediction;
- ▶ **unsupervised learning**: data-driven learning of structure such as clusters or latent representations;
- ▶ **reinforcement learning**: learning from interaction and feedback—informally, learning from the consequences of previous actions.

2.2 What It Means for a Program to Learn

Tom M. Mitchell's formulation describes learning through a **task** T , a **performance measure** P , and an **experience** E . A program learns when its measured performance on tasks in T improves with experience E .

2.2.1 Task

A machine-learning task is typically one that is difficult to solve by a conventional fixed program. A dataset contains many examples; each example is described through a collection of **features** and represented as a feature vector

$$\mathbf{x} = (x_1, x_2, \dots, x_n)^\top \in \mathbb{R}^n. \quad (2.1)$$

where

- ▶ $\mathbf{x}$: one example expressed numerically;
- ▶ x_i : the i -th feature of the example;
- ▶ n : number of features, i.e. the feature-vector dimensionality.

Typical tasks include classification, regression, synthesis/sampling, and probability estimation.

2.2.2 Performance measure

For binary classification, a confusion matrix distinguishes four outcomes:

- ▶ **TP**: true positive;
- ▶ **TN**: true negative;
- ▶ **FP**: false positive;
- ▶ **FN**: false negative.

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (2.2)$$

where TP and TN are correct predictions, while FP and FN are incorrect predictions. The denominator is the total number of evaluated examples.

Accuracy can be misleading on imbalanced data. If a dataset contains 9900 negative examples and only 100 positive examples, a classifier that predicts every sample as negative obtains 99% accuracy while identifying none of the positives.

Precision and Recall

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (2.3)$$

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (2.4)$$

where

- ▶ precision asks: among predicted positives, how many are truly positive?;
- ▶ recall asks: among truly positive examples, how many were recovered by the model?

2.2.3 Experience

In supervised learning, experience is provided by a dataset of labeled examples. Each datapoint $\mathbf{x}_i$ has a corresponding label y_i , which indicates the target class or target value associated with that datapoint.

2.3 Features, Feature Space, and Data Representation

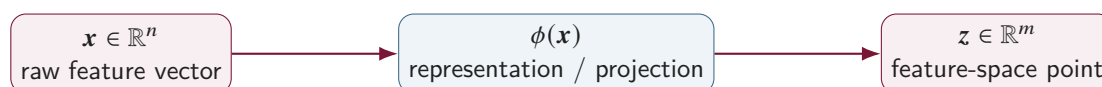
The way data are represented determines which patterns are easy or difficult for a learner to exploit. The same geometric data can look difficult to separate in Cartesian coordinates but much simpler in polar coordinates; a representation change can therefore expose useful structure.

Definition

A **feature space** is the coordinate space induced by a chosen representation or transformation of the original feature vector. There is no single universal feature space: its dimensionality and geometry depend on the transformation applied to the data.

Table 2.1: How increasingly data-driven systems obtain representations.

Approach	Representation / features	Mapping to output
Rule-based system	Hand-designed representation embedded in the program	Hand-designed rules/program
Classical machine learning	Hand-designed or engineered features	Mapping learned from data
Representation learning	Features and output mapping are learned from data	Learned mapping
Deep learning	Multiple layers learn progressively richer representations	Learned end-to-end mapping

**Figure 2.1:** A feature vector can be transformed into a different feature-space representation. Deep networks learn such transformations internally.

Deep neural networks learn feature representations rather than relying only on hand-designed features. Feature-space dimensionality depends on the learned or chosen transformation.

2.4 Linear Regression

Linear regression models the relationship between one or more explanatory variables and a dependent numerical variable. Its prediction rule is

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b. \quad (2.5)$$

where

- ▶ $\mathbf{x} \in \mathbb{R}^n$: input feature vector;
- ▶ $\mathbf{w} \in \mathbb{R}^n$: model parameters, also called weights;
- ▶ $b \in \mathbb{R}$: bias or intercept;
- ▶ $\hat{y} \in \mathbb{R}$: predicted scalar output.

A large-magnitude weight w_i means that feature x_i has a large effect on the linear prediction. A negative w_i means that increasing x_i decreases the prediction, all else being equal.

For a linear-algebra refresher, see: https://www.deeplearningbook.org/contents/linear_algebra.html.

2.4.1 Train, validation, and test sets

Standard dataset-split terminology:

- ▶ **training set**: often around 70%–90% of the dataset, used to fit model parameters;
- ▶ **test set**: held out from training and used to evaluate generalization after training;
- ▶ **validation set**: optionally held out for selecting hyperparameters during model development.

2.4.2 Mean squared error

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2. \quad (2.6)$$

where

- ▶ m : number of examples in the evaluated set;
- ▶ $y_i \in \mathbb{R}$: true target for example i ;
- ▶ $\hat{y}_i \in \mathbb{R}$: model prediction for example i .

The squared term penalizes large errors more strongly than small errors. During training the MSE can be evaluated on training data; after training it can be evaluated on held-out data to assess generalization.

2.5 Learning as Optimization

Learning can be formulated as optimization of an **objective function** (also called a criterion). A maximization problem can be converted into minimization by negating the objective, so machine-learning training is often written as a minimization problem.

A useful distinction is between **loss** and **cost**: the loss refers to the error for one example, while the cost is an average or aggregate over the dataset. In practice, papers sometimes use these terms interchangeably.

For squared error:

$$L(\hat{y}, y) = (\hat{y} - y)^2, \quad (2.7)$$

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}_i; \theta), y_i). \quad (2.8)$$

where

- ▶ L : per-example loss;
- ▶ J : dataset-level cost/objective;
- ▶ θ : all trainable model parameters;
- ▶ $f(\mathbf{x}_i; \theta)$: model prediction for example i .

2.6 Gradient-Based Optimization

2.6.1 Derivative and local linear approximation

For a scalar function $y = f(x)$, the derivative $f'(x)$ gives the instantaneous rate of change. For a sufficiently small perturbation ϵ ,

$$f(x + \epsilon) \approx f(x) + \epsilon f'(x). \quad (2.9)$$

where

- ▶ x : current input point;

- ▶ ϵ : small input perturbation;
- ▶ $f'(x)$: derivative at x .

If $f'(x) > 0$, moving in the negative direction tends to decrease f locally; if $f'(x) < 0$, moving in the positive direction tends to decrease it.

2.6.2 Single-variable gradient descent

A simple descent intuition is to move with the opposite sign of the derivative:

$$x \leftarrow x - \epsilon \operatorname{sign}(f'(x)). \quad (2.10)$$

This captures the direction of descent. Practical gradient descent typically uses the derivative magnitude as well, as in the multivariable update below.

A point where $f'(x) = 0$ is a **critical point**. Critical points include global minima, local minima, maxima, and saddle points. In high-dimensional optimization, locating a global minimum can be difficult; iterative methods generally seek useful low-loss solutions.

2.6.3 Multivariable gradients

For $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the gradient is the vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}. \quad (2.11)$$

where

- ▶ $\mathbf{x} \in \mathbb{R}^n$: optimization variable;
- ▶ $\partial f / \partial x_i$: rate of change with respect to coordinate x_i ;
- ▶ $\nabla f(\mathbf{x})$: direction of steepest local ascent.

In multiple dimensions, a differentiable critical point satisfies $\nabla f(\mathbf{x}) = \mathbf{0}$.

Gradient descent moves in the opposite direction:

$$\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x}). \quad (2.12)$$

where $\alpha > 0$ is the **learning rate**, which controls the size of each optimization step.

For ordinary linear regression, the MSE is a convex function of the linear parameters and can be solved directly by setting the gradient to zero. Neural networks generally do not admit a comparable closed-form solution, so iterative optimization is used.

2.7 Generalization, Capacity, and Overfitting

A model can achieve very low training loss while performing poorly on unseen samples. The objective is not to memorize the training set, but to learn a rule that **generalizes**.

Overfitting intuition

John von Neumann’s often-cited warning—“With four parameters I can fit an elephant, and with five I can make him wiggle his trunk”—captures the danger of excessive model flexibility: fitting the observed data is not the same as learning a rule that generalizes.

Table 2.2: Underfitting, appropriate fit, and overfitting.

Regime	Training behavior	Generalization behavior
Underfitting	Model is too restricted to capture important structure	High training and test error
Appropriate fit	Capacity is sufficient without fitting incidental noise	Low error on unseen data
Overfitting	Model fits training details/noise too aggressively	Very low training error but worse validation/test error

Capacity is the model family’s ability to fit a broad variety of functions. Increasing capacity can reduce underfitting, but excessive capacity can increase overfitting. In a polynomial model, capacity can be increased by adding higher-order features $x, x^2, x^3, \dots$

2.8 Regularization

Definition

Regularization is a modification to the learning procedure intended to reduce generalization error rather than merely reduce training error. It effectively restricts or biases the set of functions the learner prefers.

Two examples are weight decay and dataset augmentation.

2.8.1 Weight decay

A squared- ℓ_2 penalty encourages smaller parameter values. A regularized objective can be written as

$$J(\theta) = \text{MSE}_{\text{train}} + \lambda \|\theta\|_2^2. \quad (2.13)$$

where

- ▶ θ : model parameters;
- ▶ $\lambda \geq 0$: regularization coefficient;
- ▶ $\|\theta\|_2^2$: squared Euclidean norm of the parameters.

The gradient ∇J describes the local slope of the objective. The **Hessian** $H = \nabla^2 J$ contains second derivatives, so it describes how the gradient changes and captures the local curvature of the loss surface. Adding an ℓ_2 penalty contributes a positive diagonal term to this curvature and can make the optimization landscape more well-behaved.

Using the closely related convention $\frac{\lambda}{2} \|\theta\|_2^2$, the regularizer contributes

$$\nabla^2 \left(\frac{\lambda}{2} \|\theta\|_2^2 \right) = \lambda I, \quad H = H_{\text{MSE}} + \lambda I. \quad (2.14)$$

where

- H : Hessian of the total objective;
- H_{MSE} : Hessian of the unregularized MSE term;
- I : identity matrix.

Source-notation consistency

The source material uses $\lambda \|\theta\|_2^2$ on one page and $\frac{\lambda}{2} \|\theta\|_2^2$ when deriving the Hessian on the next. These conventions differ only by a constant factor that can be absorbed into the definition of λ , but the Hessian equation λI corresponds directly to the latter convention.

2.8.2 Dataset augmentation

A larger and more representative dataset usually improves generalization, but real data are limited. **Dataset augmentation** creates additional reasonable examples.

- **Synthetic data**: new examples generated from scratch, potentially by another generative model.
- **Augmented data**: transformations of existing examples x_i that preserve the relevant semantics.

For images, common augmentation examples here are noise injection, geometric transformations, color transformations, erasure, cropping, flipping, rotation, and brightness adjustment.

2.9 Data Normalization

Features can have very different scales, such as age and income. Large scale differences can skew a DNN's loss surface, slow gradient-based optimization, cause updates to overshoot useful optima or become poorly directed, and increase numerical-instability risk. Normalization rescales features to a comparable range, typically producing a smoother optimization surface and making gradient-based learning easier.

2.9.1 Min-max scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}. \quad (2.15)$$

where

- x : original feature value;
- $x_{\min}$ and $x_{\max}$: minimum and maximum values used for scaling;
- x' : rescaled value, typically in $[0, 1]$.

Min-max scaling is sensitive to outliers.

2.9.2 Z-score standardization

$$x' = \frac{x - \mu}{\sigma}. \quad (2.16)$$

where

- ▶ μ : feature mean;
- ▶ σ : feature standard deviation;
- ▶ x' : standardized value, with approximately zero mean and unit standard deviation when computed over the same dataset.

2.10 Validation and Cross-Validation

A validation set is held out during training and used to evaluate the model at intermediate stages, tune hyperparameters such as the learning rate or number of parameters, and detect overfitting. A pattern of high training performance but low validation performance is evidence of poor generalization.

2.10.1 K-fold cross-validation

The data are split into K folds. For each run, the model is trained on $K - 1$ folds and evaluated on the remaining fold. The process is repeated until each fold has served as the held-out fold. Depending on the experimental protocol, the held-out fold can support validation/model selection or performance testing/estimation.

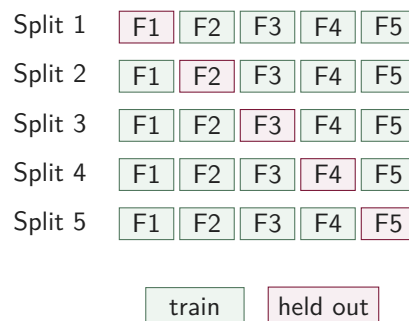


Figure 2.2: Schematic 5-fold cross-validation. Each fold is held out once.

Cross-validation often gives a better approximation of model performance, reduces sampling bias, and maximizes data utilization, but it is computationally expensive because training is repeated multiple times.

Key Takeaways

- ▶ ML examples are represented as feature vectors, and representation strongly affects learnability.
- ▶ Accuracy alone can be deceptive, especially with class imbalance; precision and recall expose different error modes.
- ▶ Linear regression predicts with $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$ and can be trained by minimizing MSE.
- ▶ Gradient descent uses local derivative information to iteratively reduce an objective.
- ▶ Generalization, not training error alone, is the objective; capacity and regularization control the fit/generalization tradeoff.
- ▶ Validation sets and cross-validation are tools for model selection and reliable performance estimation.

Decision Trees, Random Forests, and Deep Neural Networks

3.1 Decision Trees

A decision tree is a hierarchical collection of conditions, also called splits or tests. Conditions are based on the input features and the decision criteria are learned from data. A condition can be **axis-aligned**, involving one feature, or **oblique**, involving multiple features.

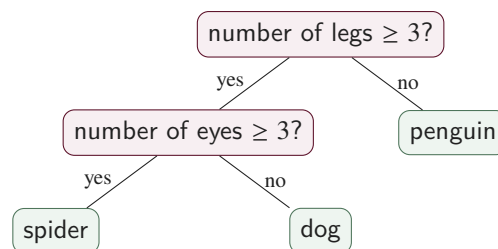


Figure 3.1: A small binary decision tree: internal nodes evaluate conditions and leaves provide predictions.

Binary decision trees use two-way conditions and are commonly used as the base learners of decision forests such as Random Forests. Non-binary trees can branch into a variable number of conditions, but excessive branching can contribute to overfitting.

3.1.1 Training a decision tree

Finding an optimal decision tree is NP-hard, so practical training relies on heuristic methods such as greedy divide-and-conquer:

1. start at the root node;
2. evaluate candidate conditions;
3. select the “best” condition according to a chosen metric;
4. recursively repeat the process on the resulting child nodes;
5. stop when a newly created node contains only, or almost only, samples from one class, i.e. it has high **purity**.

The set of possible conditions can be very large or even effectively infinite for continuous features, so additional heuristics are used to generate plausible split candidates.

3.1.2 Entropy as a purity measure

Entropy can be used as a measure of class disorder:

$$H(\mathbf{p}) = - \sum_{i=1}^C p_i \log_2 p_i. \quad (3.1)$$

where

- ▶ C : number of classes;
- ▶ p_i : fraction of samples belonging to class i at the node, with $\sum_{i=1}^C p_i = 1$;
- ▶ H : entropy of the node's class distribution.

Notation Note

The source slide denotes the class fractions by y_i . The symbol p_i is used here to avoid confusing class proportions with the target labels y_i used elsewhere in the notes.

Entropy is highest when classes are strongly mixed (for two balanced classes, a 50/50 mixture) and is minimal when the node contains samples from only one class.

Information Gain

Information gain measures the expected reduction in entropy produced by a split on feature f :

$$G(D, f) = H(D) - \sum_{v \in \text{values}(f)} \frac{|D_v|}{|D|} H(D_v). \quad (3.2)$$

where

- ▶ D : set of samples at the current node;
- ▶ f : candidate feature;
- ▶ v : a value of feature f ;
- ▶ $D_v \subseteq D$: samples in D whose feature f has value v ;
- ▶ $|D_v|/|D|$: relative size of the child subset;
- ▶ $H(D)$ and $H(D_v)$: parent and child entropies.

The second term is the entropy of each possible resulting subset, weighted by its relative size.

For discrete features, candidate values can be evaluated directly and the split with the largest information gain selected. In the simplified treatment used here, continuous features are discretized before applying the same idea. The procedure is then applied recursively until the available selection features or stopping criteria are exhausted.

3.2 Ensemble Methods and Random Forests

3.2.1 Ensemble learning

An ensemble combines multiple classifiers into a single predictor. Each learner estimates the target function, and the individual predictions are aggregated. Combining **independent and diverse** decisions generally reduces variance and can reduce the final error rate relative to a single model.

3.2.2 Random Forest construction

A Random Forest is an ensemble of decision trees with two principal sources of randomization:

1. **bootstrap sampling**: for each tree, randomly sample training instances with replacement;
2. **random feature selection**: at each decision node, choose the best split only from a random subset of m features rather than from all available features. In the source terminology, this is the **random vector method**.

Each bootstrapped sample is used to train an unpruned decision tree. Samples not selected for a particular bootstrap sample can be used to estimate that tree's error. For prediction, the individual tree outputs are aggregated, usually by majority voting for classification, although weighted aggregation is also possible.

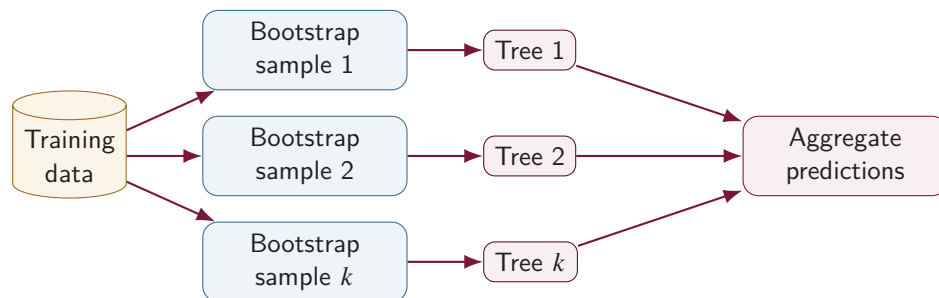


Figure 3.2: Random Forest: bootstrapped datasets train diverse trees whose predictions are aggregated. Each tree also uses random feature subsets at its decision nodes.

3.2.3 Why Random Forests work well

Random Forests have several useful properties:

- ▶ bootstrapping and random feature selection promote independence among the decision trees;
- ▶ greater independence is a key requirement for effective ensembles;
- ▶ increasing the number of diverse trees is less likely to overfit than repeatedly relying on the same single-tree structure;
- ▶ predictions depend less strongly on any one specific training sample;
- ▶ Random Forests are often computationally cheaper than complex neural networks while still performing well on many tasks.

3.3 Deep Neural Networks

3.3.1 What “deep” means

Deep Neural Network (DNN) is an umbrella term for neural networks with many layers. Examples include deep feedforward networks, convolutional neural networks, recurrent neural networks, generative adversarial networks, and transformer-based neural networks.

The “neural” terminology comes from an analogy between an artificial computational unit and a biological neuron: multiple inputs are weighted, combined, and passed through a function that produces an output.

3.3.2 Function approximation

A DNN can be viewed as a function approximator. Suppose the unknown target relationship is

$$y = f^*(\mathbf{x}). \quad (3.3)$$

A neural network defines an approximation

$$\hat{y} = f(\mathbf{x}; \theta), \quad (3.4)$$

where

- ▶ $\mathbf{x}$: input feature vector;
- ▶ f^* : unknown target function;
- ▶ $f(\cdot; \theta)$: neural-network approximation parameterized by θ ;
- ▶ y : target output and $\hat{y}$: network prediction.

Training adjusts θ so that $f(\mathbf{x}; \theta)$ approximates $f^*(\mathbf{x})$ on the relevant data distribution.

3.3.3 Feedforward composition and depth

A feedforward network composes several functions in a directed acyclic graph. For three layers,

$$f(\mathbf{x}) = f^3\left(f^2\left(f^1(\mathbf{x})\right)\right). \quad (3.5)$$

where

- ▶ f^1, f^2, f^3 : transformations implemented by consecutive network layers;
- ▶ the **depth** of the network is the length of this composition;
- ▶ feedforward means information moves forward through the directed acyclic computation graph.

Training examples provide only inputs and desired outputs. For each datapoint $\mathbf{x}_i$, the label satisfies conceptually

$$y_i = f^*(\mathbf{x}_i), \quad (3.6)$$

and the learning algorithm modifies the hidden-layer parameters so the network output approaches y_i .

3.4 Representation Learning in Feedforward Networks

A one-hidden-layer network can be written as a learned nonlinear transformation followed by a linear map:

$$\hat{y} = f(\mathbf{x}; \theta, \mathbf{w}) = \phi(\mathbf{x}; \theta)^\top \mathbf{w}. \quad (3.7)$$

where

- ▶ $\phi(\mathbf{x}; \theta)$: nonlinear representation or feature extractor;
- ▶ θ : parameters controlling the learned representation;
- ▶ $\mathbf{w}$: parameters mapping the learned representation to the output;
- ▶ $\hat{y}$: network prediction.

This is analogous to linear regression $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$, except that the linear mapping is applied to a *learned* representation rather than directly to the raw input. The representation ϕ maps the input into an intermediate feature space useful for the task.

The benefit is the ability to learn both a nonlinear feature transformation and an output mapping from a broad family of functions. The cost is that the optimization problem is generally non-convex and lacks a simple closed-form solution.

For compactness, all trainable parameters can be absorbed into a single symbol:

$$\hat{y} = f(\mathbf{x}; \theta) = f(\mathbf{x}; \mathbf{W}, \phi), \quad (3.8)$$

where θ denotes all parameters that define the learned network function.

3.5 Understanding the Neuron and Activation Function

3.5.1 Weighted inputs and a threshold

A music-festival decision provides an intuitive neuron example: weather, friends, and transport are features x_1, x_2, x_3 , their importance is encoded by weights w_1, w_2, w_3 , and an activation function g converts the weighted combination into a decision.

A threshold unit can be written as

$$g(\mathbf{x}) = \begin{cases} 1, & \sum_i x_i w_i \geq \text{threshold}, \\ 0, & \sum_i x_i w_i < \text{threshold}. \end{cases} \quad (3.9)$$

Equivalently, by absorbing the threshold into a bias b ,

$$g(\mathbf{x}) = \begin{cases} 1, & \mathbf{w}^\top \mathbf{x} + b \geq 0, \\ 0, & \mathbf{w}^\top \mathbf{x} + b < 0. \end{cases} \quad (3.10)$$

where

- ▶ $\mathbf{x}$: input features;
- ▶ $\mathbf{w}$: feature weights;
- ▶ b : bias, which shifts the threshold;

- g : activation/decision function.

Changing $\mathbf{w}$ changes the relative importance of the features. In modern neural networks, the activation is usually a differentiable or piecewise-differentiable nonlinear function rather than a hard threshold.

3.5.2 Activation functions

Let

$$z = \mathbf{w}^\top \mathbf{x} + b. \quad (3.11)$$

where $\mathbf{x}$ is the input, $\mathbf{w}$ the weight vector, b the bias, and z the scalar pre-activation. The activation function g maps z to the neuron's output.

Linear activation

Consider the simplified linear activation $g(z) = \lambda z$. With $z = \mathbf{w}^\top \mathbf{x} + b$,

$$\hat{y} = g(z) = \lambda (\mathbf{w}^\top \mathbf{x} + b). \quad (3.12)$$

If every layer uses only linear activations, the entire composition remains linear. Therefore, multiple layers do not create a richer nonlinear function family.

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (3.13)$$

The sigmoid smoothly approximates a step and maps real-valued inputs to $(0, 1)$. Its smoothness makes it useful for illustrating gradient-based learning and backpropagation.

ReLU

$$\text{ReLU}(z) = \max\{0, z\}. \quad (3.14)$$

ReLU is piecewise linear and widely used in feedforward networks. It preserves simple gradient behavior over its active region while introducing nonlinearity into the overall composition.

Softmax

For a vector of logits $\mathbf{z}$, the softmax output for coordinate i is

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}. \quad (3.15)$$

where

- z_i : score/logit for class i ;
- the denominator sums exponentiated scores over all classes;
- the output vector is normalized and can be interpreted as a probability distribution.

Softmax is therefore useful for multi-class classification outputs.

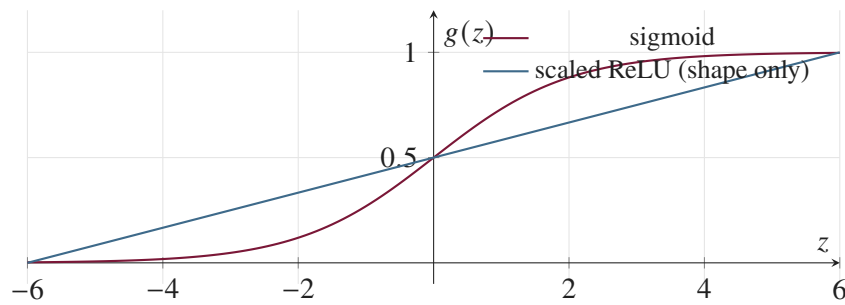


Figure 3.3: Schematic activation-function shapes. ReLU is scaled only to share the panel; the analytic definition in equation (3.14) is authoritative.

3.6 Forward Propagation

A layer first computes an affine transformation and then applies an activation:

$$\mathbf{z}^i = \mathbf{W}^i \mathbf{a}^{i-1} + \mathbf{b}^i, \quad (3.16)$$

$$\mathbf{a}^i = g(\mathbf{z}^i). \quad (3.17)$$

where

- i : layer index;
- $\mathbf{W}^i$: weight matrix of layer i ;
- $\mathbf{b}^i$: bias vector of layer i ;
- $\mathbf{a}^{i-1}$: activations from the previous layer, with $\mathbf{a}^0 = \mathbf{x}$;
- $\mathbf{z}^i$: pre-activation vector;
- $\mathbf{a}^i$: activation vector.

For a two-layer example:

$$\mathbf{z}^1 = \mathbf{W}^1 \mathbf{x} + \mathbf{b}^1, \quad \mathbf{a}^1 = \sigma(\mathbf{z}^1), \quad (3.18)$$

$$\mathbf{z}^2 = \mathbf{W}^2 \mathbf{a}^1 + \mathbf{b}^2, \quad \hat{y} = \mathbf{a}^2 = \sigma(\mathbf{z}^2). \quad (3.19)$$

Dimension check for the worked forward pass

Using the column-vector convention of equation (3.17), let $\mathbf{x} \in \mathbb{R}^3$, use three hidden units, and produce one scalar output. Then

$$\mathbf{W}^1 \in \mathbb{R}^{3 \times 3}, \quad \mathbf{b}^1 \in \mathbb{R}^3, \quad \mathbf{z}^1, \mathbf{a}^1 \in \mathbb{R}^3,$$

$$\mathbf{W}^2 \in \mathbb{R}^{1 \times 3}, \quad \mathbf{b}^2 \in \mathbb{R}, \quad \mathbf{z}^2, \mathbf{a}^2 = \hat{y} \in \mathbb{R}.$$

Thus $\mathbf{W}^1 \mathbf{x}$ is a 3-vector and $\mathbf{W}^2 \mathbf{a}^1$ is a scalar. The source slide displays an equivalent row-vector/transposed convention for some shapes; this document uses column vectors consistently throughout.

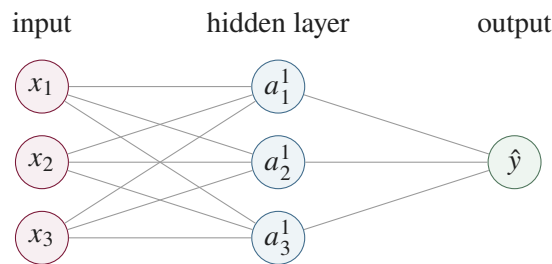


Figure 3.4: Forward propagation computes each layer from the previous layer until the network output $\hat{y}$ is obtained.

3.7 Gradient Descent for Neural Networks

The network parameters θ consist of all weight matrices and bias vectors. For a dataset-level objective

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}_i; \theta), y_i), \quad (3.20)$$

we iteratively update parameters in the direction that reduces the objective:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} J(\theta). \quad (3.21)$$

where

- m : number of examples used to evaluate the objective;
- L : per-example loss;
- θ : all network parameters;
- α : learning rate.

For an individual layer i , define

$$dW^i = \frac{\partial J}{\partial W^i}, \quad d\mathbf{b}^i = \frac{\partial J}{\partial \mathbf{b}^i}, \quad (3.22)$$

$$W^i \leftarrow W^i - \alpha dW^i, \quad \mathbf{b}^i \leftarrow \mathbf{b}^i - \alpha d\mathbf{b}^i. \quad (3.23)$$

where dW^i and $d\mathbf{b}^i$ denote the gradients of the objective with respect to the weights and biases of layer i ; α is the learning rate.

3.8 Backpropagation

Backpropagation efficiently computes the gradients needed by equation (3.23). The chain-rule argument can be organized into four central relationships.

3.8.1 Error at the output layer

For a network with N layers, $\mathbf{a}^i = g(\mathbf{z}^i)$ and $\hat{\mathbf{y}} = \mathbf{a}^N$. The error with respect to the final pre-activation is

$$d\mathbf{z}^N = \nabla_{\mathbf{z}^N} J = \nabla_{\mathbf{a}^N} J \frac{\partial \mathbf{a}^N}{\partial \mathbf{z}^N}. \quad (3.24)$$

where

- $\nabla_{\mathbf{a}^N} J$: gradient of the objective with respect to the network output;
- $\partial \mathbf{a}^N / \partial \mathbf{z}^N$: Jacobian of the output activation function;
- $d\mathbf{z}^N$: output-layer error signal with respect to the pre-activation.

The **Jacobian** generalizes a gradient to a vector-valued function $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$; it is the matrix containing all partial derivatives of the outputs with respect to the inputs.

3.8.2 Propagating error to an earlier layer

Recall that

$$\mathbf{z}^{i+1} = W^{i+1} \mathbf{a}^i + \mathbf{b}^{i+1}. \quad (3.25)$$

Using the affine relation above,

$$d\mathbf{a}^i = \nabla_{\mathbf{a}^i} J = \left(W^{i+1} \right)^\top d\mathbf{z}^{i+1}. \quad (3.26)$$

Intuitively, the error at layer i is obtained by transporting the downstream error backward through the weights that connect layer i to layer $i + 1$.

Combining equation (3.26) with the Jacobian of the activation at layer i gives $d\mathbf{z}^i$, and the process repeats backward until the first hidden layer.

3.8.3 Weight and bias gradients

Once $d\mathbf{z}^i$ is known,

$$dW^i = d\mathbf{z}^i \left(\mathbf{a}^{i-1} \right)^\top, \quad (3.27)$$

$$d\mathbf{b}^i = d\mathbf{z}^i. \quad (3.28)$$

where

- dW^i : gradient of the objective with respect to the layer- i weights;
- $d\mathbf{b}^i$: gradient with respect to the layer- i bias;
- $\mathbf{a}^{i-1}$: input activation vector to layer i .

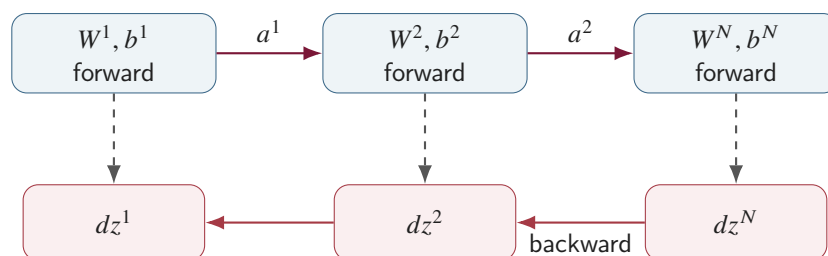


Figure 3.5: Backpropagation computes output error first and then propagates derivative information backward through the network.

Backpropagation summary

The four central relationships are:

$$\begin{aligned} dz^N &= \nabla_{\mathbf{a}^N} J \frac{\partial \mathbf{a}^N}{\partial \mathbf{z}^N}, \\ d\mathbf{a}^i &= (W^{i+1})^\top dz^{i+1}, \\ dW^i &= dz^i (\mathbf{a}^{i-1})^\top, \\ d\mathbf{b}^i &= dz^i. \end{aligned}$$

They are an application of the chain rule to the layered computation graph.

3.9 Stochastic Gradient Descent

Computing gradients over a very large entire dataset at every update can be prohibitively expensive.

Gradient-descent terminology

Full-batch gradient descent uses the entire training set for one update. **Pure stochastic gradient descent** uses one example per update. The procedure used here samples a subset of examples and is therefore **mini-batch SGD**, the standard practical variant for neural-network training.

The mini-batch procedure is:

1. randomly select a subset of the training data, called a **mini-batch**, of size m ;
2. compute gradients for the examples in the mini-batch;
3. update weights and biases;
4. continue until the training data have been traversed;
5. repeat for the desired number of **epochs**.

Epoch and mini-batch

A **mini-batch** is the subset of examples used for one gradient computation/update. An **epoch** is one traversal of the training dataset under the batching scheme.

3.10 Loss Functions

Common losses include MSE, binary cross-entropy, cross-entropy, contrastive/triplet losses, and Kullback–Leibler divergence. The first three are developed here; contrastive/triplet losses and KL divergence are listed for completeness but are not developed further.

3.10.1 Mean Squared Error

For regression:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2. \quad (3.29)$$

where

- m : number of evaluated examples;
- y_i : true target for example i ;
- $\hat{y}_i$: prediction for example i .

MSE penalizes large errors more strongly because the error is squared.

3.10.2 Binary cross-entropy

For binary classification,

$$\mathcal{L}_{\text{BCE}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]. \quad (3.30)$$

where

- $y \in \{0, 1\}$: true binary label;
- $\hat{y} \in (0, 1)$: predicted probability of class 1;
- $1 - \hat{y}$: predicted probability of class 0.

BCE is naturally paired with a sigmoid output layer so that the prediction lies in $(0, 1)$ and can be interpreted as a Bernoulli probability.

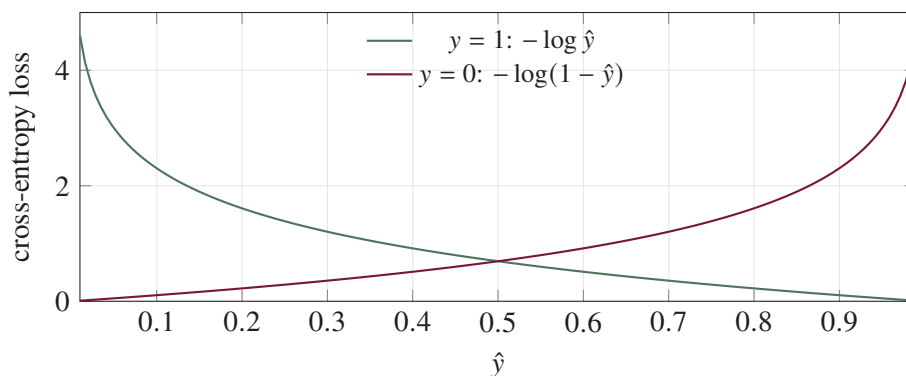


Figure 3.6: Binary cross-entropy penalizes confident wrong predictions sharply. These curves follow equation (3.30) exactly.

3.10.3 Multi-class cross-entropy

For C classes,

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^C y_i \log(\hat{y}_i). \quad (3.31)$$

where

- C : number of classes;
- y_i : target probability/indicator for class i ;
- $\hat{y}_i$: predicted probability for class i .

Multi-class cross-entropy is naturally paired with a softmax output layer, which produces a normalized class-probability vector.

Key Takeaways

- ▶ Decision-tree learning greedily chooses splits that increase class purity; entropy and information gain provide one split criterion.
- ▶ Random Forests combine bootstrap sampling and random feature subsets to create diverse trees and reduce variance.
- ▶ A DNN is a composition of many parameterized transformations; hidden layers learn intermediate representations.
- ▶ Nonlinear activation functions are essential: a composition of only linear layers is still linear.
- ▶ Forward propagation computes activations; backpropagation applies the chain rule to compute gradients layer by layer in reverse.
- ▶ Mini-batch stochastic gradient descent makes large-scale neural-network optimization computationally feasible.
- ▶ The output activation and loss function should match the task: MSE for regression, sigmoid+BCE for binary classification, and softmax+cross-entropy for multi-class classification.

Notation and Quick Reference

A.1 Core Symbols

Table A.1: Notation used throughout the notes.

Symbol	Meaning
$\mathbf{x}$	Input feature vector
y	True target / label
$\hat{y}$	Model prediction
$\mathbf{w}, W$	Weight vector / weight matrix
$b, \mathbf{b}$	Bias scalar / bias vector
θ	Collection of trainable model parameters
L	Per-example loss
J	Dataset-level objective / cost
α	Learning rate
∇	Gradient operator
ϕ	Learned or chosen feature transformation
$\mathbf{z}^i$	Pre-activation at layer i
$\mathbf{a}^i$	Activation at layer i

A.2 Formula Summary

Linear regression: $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$

MSE: $\text{MSE} = \frac{1}{m} \sum_i (\hat{y}_i - y_i)^2$

Gradient descent: $\theta \leftarrow \theta - \alpha \nabla_\theta J$

Entropy: $H = - \sum_i p_i \log_2 p_i$

Information gain: $G(D, f) = H(D) - \sum_v \frac{|D_v|}{|D|} H(D_v)$

Sigmoid: $\sigma(z) = \frac{1}{1 + e^{-z}}$

ReLU: $\text{ReLU}(z) = \max\{0, z\}$

Softmax: $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$

BCE: $\mathcal{L}_{\text{BCE}} = - [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$

Cross-entropy: $\mathcal{L}_{\text{CE}} = - \sum_i y_i \log \hat{y}_i.$

Glossary

Adversarial example	An input deliberately manipulated to induce undesired model behavior while remaining feasible for the application.
Backpropagation	Reverse-mode application of the chain rule used to compute neural-network parameter gradients.
Black-box attack	In the simplified taxonomy used here, an attack with no internal model knowledge.
Capacity	Ability of a model family to represent a wide variety of functions.
Epoch	One traversal of the training dataset under the current batching procedure.
Feature	Measurable or derived component used to represent an example.
Feature space	Space induced by a chosen or learned representation of feature vectors.
Gray-box attack	Attack with partial knowledge of the target system.
Information gain	Expected reduction in entropy resulting from a candidate decision-tree split.
Mini-batch	Subset of training examples used for one gradient computation/update.
Overfitting	Learning training-specific detail that does not generalize well to unseen samples.
Poisoning	Training-time manipulation of learning data intended to alter the resulting model.
Regularization	Modification of a learning procedure intended to reduce generalization error.
Threat model	Explicit specification of adversarial knowledge, goals, and capabilities.
White-box attack	Attack under perfect or near-perfect knowledge of the target model and relevant data in the simplified taxonomy used here.

Source Coverage Map

This appendix records the result of the final completeness audit. Slide ranges refer to the three supplied source PDFs. Status refers to *substantive intellectual content*: title-only slides, repeated screenshots, and decorative examples may be condensed when their security or ML lesson is already represented explicitly.

Course Intro / Threat Modeling

Slides	Status	Representation in these notes
1–9	Fully represented	Orientation, logistics, assessment, and tentative syllabus.
10–21	Condensed / visual content summarized	Voice assistants, autonomous driving, adversarial traffic signs/audio, LLM jailbreaks, prompt injection, browser/agent risks, and economic asymmetry are preserved in prose.
22–31	Fully represented / recreated diagrams	Classical vs. adversarial world models, assumptions, and the spam-classifier example.
32–34	Fully represented / recreated diagram	Adversarial world model, arms race, and key rules.
35–43	Fully represented	Threat-model dimensions, CIA goals, white/gray/black-box knowledge, and Kerckhoffs' principle.
44–52	Fully represented	Training/inference capabilities, Microsoft Tay, feasibility constraints, and realistic capability levels.

Basic ML Refresher

Slides	Status	Representation in these notes
1–9	Fully represented	ML paradigms, $T/P/E$, task types, confusion-matrix outcomes, accuracy, precision, recall, and experience.
10–14	Condensed / recreated diagram	Representation, Cartesian/polar intuition, feature-space projection, and learned features.
15–20	Fully represented	Linear regression, weights/bias, train/validation/test terminology, and MSE.
21–31	Fully represented	Objective/loss/cost, derivatives, critical points, gradients, learning rate, and linear-regression learning.
32–37	Condensed / visual examples summarized	Overfitting/underfitting, generalization, and capacity; illustrative plots are summarized rather than copied.
38–43	Fully represented / visual examples summarized	Regularization, ℓ_2 weight decay with Hessian intuition, and dataset augmentation.
44–47	Fully represented / recreated diagram	Normalization, validation, and K -fold cross-validation.

Decision Trees, RFs, and DNNs

Slides	Status	Representation in these notes
1–9	Fully represented / recreated diagram	Decision trees, binary/non-binary branching, training, entropy, and information gain.
10–14	Fully represented / recreated diagram	Ensemble learning and Random Forest construction/advantages.
15–28	Condensed / recreated concepts	DNN overview, biological analogy, function approximation, feedforward composition, representation learning, and notation.
29–37	Fully represented / recreated plot	Neuron intuition, thresholds/biases, linear activation, sigmoid, ReLU, and softmax.
38–45	Fully represented / worked dimension check	Forward propagation, parameter shapes, objective, and gradient-descent updates.
46–53	Fully represented / recreated diagram	Chain-rule backpropagation, output error, Jacobian, hidden-layer error, dW , and db .
54–61	Fully represented / recreated plot	Mini-batch SGD, epoch definition, MSE, BCE, cross-entropy, and the additional loss names listed in the source.